30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 27

Advanced Architecture — State Machines, Plugins, Server-Driven UI & Evolving Large Apps



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27

Why advanced architecture matters in 2026

Day 3 was one screen and one app. Architect interviews ask what happens when **many teams, several platforms and years of releases** hit that architecture:

- Many teams ship **one binary** → ownership, contracts, plugins
- Users run **old app versions** for a long time → backward-compatible APIs, server-driven UI, flags
- **Complex flows** (payments, calls, onboarding) → explicit state machines
- **Bad networks and live data** → one offline-first pipeline
- Decisions **outlive their authors** → ADRs and migration strategies

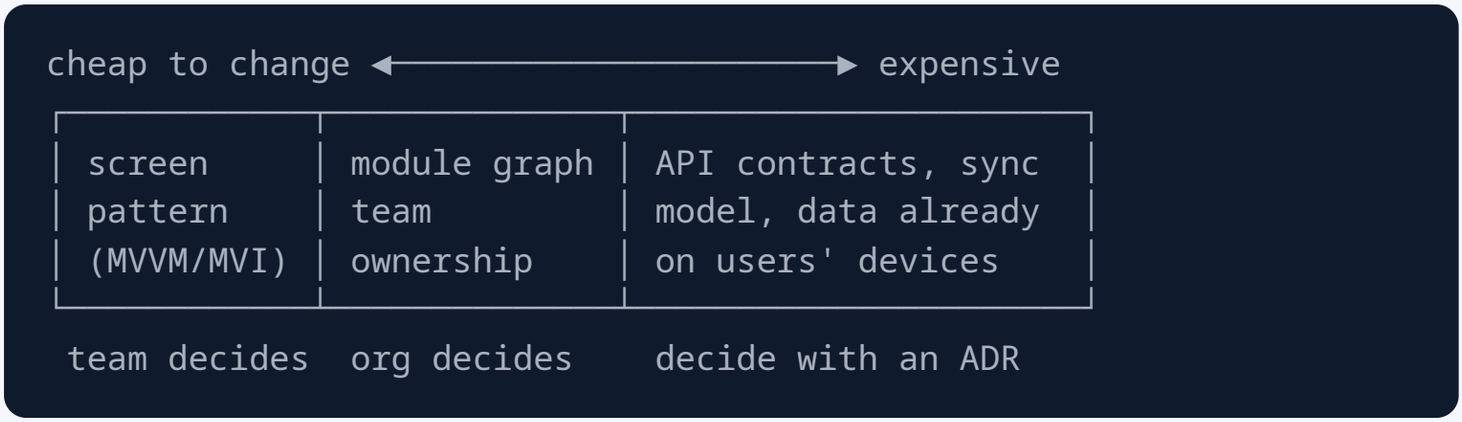
Level	Questions
Junior	What changes at scale · Domain modeling · Events vs commands vs state
Mid	State machines · Plugin architecture · Cross-cutting concerns · CQRS & event sourcing on a phone
Senior	Server-driven UI · Feature teams & "micro-frontends" · Offline-first & real-time at product level
Architect	Decision making & ADRs · Evolving architectures · The app in a distributed system



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27

Mental model: spend architecture where change is expensive



- **Architecture = the decisions that are expensive to reverse** — not the folder structure
- Screen patterns (Day 3) are cheap to change; contracts with the backend and data on millions of devices are not
- Put rigor where reversal is expensive; keep everything else easy to change

THE ARCHITECT'S QUESTION

"What will be hardest to change in two years — and how do we keep that option open today?"



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27

Junior

What changes at scale · Domain modeling · Events vs commands vs state



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What changes when an app gets big

Force	Architectural answer
Many teams, one binary	Module ownership, `:api` contracts (Day 25), plugins (Q5)
Users update late	Backward-compatible APIs, server-driven UI, flags (Q8, Q13)
Complex multi-step flows	Explicit state machines (Q4)
Bad networks, live data	One offline-first pipeline (Q10)
A codebase that lives for years	ADRs and incremental migration (Q11, Q12)

Conway's law: systems mirror the communication structure of the organization that builds them. Module boundaries end up following team boundaries — so design both together.

INTERVIEW TRAP

"Large-scale architecture means more layers." — Layers add indirection, not scalability. What scales is clear ownership, stable contracts and independent change.

INTERVIEW TRAP

"In Clean Architecture, data is an inner layer and presentation is the outermost one." (last year's description) — UI and data are **both outer** adapters that depend on the domain (Day 3).



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Domain modeling in Kotlin

```kotlin
@JvmInline
value class Money(val minor: Long) { // smallest currency unit
    init { require(minor >= 0) { "Negative amount" } }
    operator fun plus(other: Money) = Money(minor + other.minor)
}

@JvmInline value class OrderId(val raw: String)

data class Order(val id: OrderId, val lines: List<Line>) {
    init { require(lines.isNotEmpty()) { "Empty order" } }

    val total: Money
        get() = lines.fold(Money(0)) { sum, l -> sum + l.price }

    fun add(line: Line): Order = copy(lines = lines + line)
}
```

- **Value objects** (`Money`, `OrderId`): no identity, validated on creation; a `value class` usually costs no allocation
- **Entities and aggregates** (`Order`): identity plus invariants; `copy()` goes through the constructor, so `init` checks still run
- An `OrderId` can't be passed where a `UserId` is expected — the compiler catches it



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Bounded contexts and traps

Bounded context: one model per part of the business. "Product" in the catalog (description, images) is not "Product" in the cart (price, quantity). Each context maps naturally to a feature module (Day 25).

INTERVIEW TRAP

"Money is a `Double`." — Binary floating point can't represent 0.1 exactly; totals drift by rounding errors. Use the smallest currency unit as a `Long` (or `BigDecimal`) inside a value class.

INTERVIEW TRAP

"One shared `Product` model in `:core:model` for every feature." — Every feature's needs leak into it, and every change rebuilds every module. Each context owns its model; map at the boundary.

FOLLOW-UP

"Why validate on the client if the server validates anyway?" — The server stays the authority. Client-side invariants give instant feedback and keep impossible states out of the UI.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Events, commands and state

Kind	Means	Example	Kotlin tool
Command	"Do X" — can be rejected, one handler	`PlaceOrder`	`suspend fun`, an outbox entry
Event	"X happened" — a fact, zero or more listeners	`OrderPlaced`	`SharedFlow`, a persisted log
State	"What is true now" — only the latest value matters	`cartCount = 3`	`StateFlow`, a Room `Flow`

```
// "User logged in" is state, not an event
interface SessionRepository {
    val session: StateFlow<Session>
}

// A late subscriber still gets the current value
sessionRepository.session
    .map { it.userId }
    .distinctUntilChanged()
    .onEach { userId -> analytics.setUserId(userId) }
    .launchIn(appScope)
```

INTERVIEW TRAP

"Publish `UserLoggedIn` on a global `SharedFlow` event bus." (last year's example) — With `replay = 0`, a subscriber that starts late — analytics initialized after login — never sees it. Login is **state**: expose it from its owner.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27



Mid-Level

State machines · Plugin architecture · Cross-cutting concerns · CQRS & event sourcing on a phone



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. State machines for complex flows

```
sealed interface PayState {
    data object Idle : PayState
    data class Paying(val orderId: String) : PayState
    data class Paid(val receipt: String) : PayState
    data class Failed(val orderId: String) : PayState
}
sealed interface PayEvent {
    data class Pay(val orderId: String) : PayEvent
    data class Approved(val receipt: String) : PayEvent
    data object Declined : PayEvent
    data object Retry : PayEvent
}

fun PayState.on(e: PayEvent): PayState = when {
    this is Idle && e is Pay -> Paying(e.orderId)
    this is Paying && e is Approved -> Paid(e.receipt)
    this is Paying && e == Declined -> Failed(orderId)
    this is Failed && e == Retry -> Paying(orderId)
    else -> this // not allowed in this state: ignore
}
```

Explicit states, explicit **allowed transitions**, everything else rejected. A double tap on "Pay" while `Paying` does nothing.

INTERVIEW TRAP

"An MVI reducer is already a state machine." — Only if it rejects events that are invalid in the current state. A reducer that accepts every intent everywhere is just a big `when`; a state machine's value is the transitions it **forbids**.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Running the machine

```
class PaymentViewModel(
    private val api: PaymentApi,
) : ViewModel() {
    private val _state = MutableStateFlow<PayState>(Idle)
    val state: StateFlow<PayState> = _state.asStateFlow()

    fun send(e: PayEvent) { // called on the main thread
        val next = _state.value.on(e)
        if (next == _state.value) return
        _state.value = next
        if (next is Paying) authorize(next.orderId)
    }

    private fun authorize(id: String) = viewModelScope.launch {
        when (val r = api.authorize(id)) { // sealed result
            is AuthResult.Ok -> send(Approved(r.receipt))
            AuthResult.Declined -> send(Declined)
        }
    }
}
```

- Side effects start **on entering a state**; their results come back as events. `on()` stays pure — test every edge without mocks

FOLLOW-UP

"The process dies mid-payment?" — Persist the state and order id (`SavedStateHandle` or Room); on restart, ask the server for the status — never re-send "Pay" blindly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Plugin architecture: extension points

```
// :core:home:api — the extension point
interface HomeSection {
    val key: String
    val order: Int
    @Composable fun Content(modifier: Modifier)
}

// :feature:loyalty:impl — one plugin
class LoyaltySection @Inject constructor() : HomeSection {
    override val key = "loyalty"
    override val order = 20
    @Composable override fun Content(modifier: Modifier) {
        LoyaltyCard(modifier)
    }
}

@Module @InstallIn(SingletonComponent::class)
interface LoyaltyModule {
    @Binds @IntoSet fun section(s: LoyaltySection): HomeSection
}
```

The host defines the interface; features plug in through DI multibinding (`@IntoSet`, `@IntoMap`). The host never imports a feature.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. The host side

```
// :feature:home:impl — knows only the interface
class HomeViewModel @Inject constructor(
    all: Set<@JvmSuppressWildcards HomeSection>,
    flags: FeatureFlags,
) : ViewModel() {
    val sections: List<HomeSection> = all
        .filter { flags.isEnabled(it.key) }
        .sortedBy { it.order }
}
```

- Typical extension points: home sections, settings entries, deep-link handlers, startup tasks, analytics sinks, payment methods
- Adding a feature = adding a module; removing it = deleting one dependency line in `:app`

INTERVIEW TRAP

"Plugins run in the order their modules were added." — A multibound `Set` has no ordering contract. Sort by an explicit key.

INTERVIEW TRAP

"Wrap each plugin's UI in try/catch so one bad plugin can't crash home." — Compose doesn't allow try/catch around composable calls. Validate plugin data in the ViewModel, keep plugin UI simple, and gate risky plugins behind a remote flag.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Cross-cutting concerns

Concern	Put it in
Auth headers, logging, retries	HTTP interceptors or Ktor plugins (Day 11)
Timing and caching around a repository	A decorator, bound through DI
Screen analytics	One navigation listener — not code in every screen
Logging and crash context	A `Logger` interface with pluggable sinks
Feature flags, remote config	An injected `FeatureFlags` interface

INTERVIEW TRAP

"Put logging, analytics and error handling in a `BaseActivity` or `BaseViewModel`." — Inheritance turns cross-cutting concerns into a god base class: every screen must extend it, nobody can opt out, and you only get one superclass. Compose behavior instead.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. A decorator instead of inheritance

```
class TimedFeedRepository(
    private val delegate: FeedRepository,
    private val metrics: Metrics,
) : FeedRepository by delegate {
    override suspend fun refresh() {
        val mark = TimeSource.Monotonic.markNow()
        try {
            delegate.refresh()
        } finally {
            metrics.record("feed_refresh", mark.elapsedNow())
        }
    }
}
```

- `by delegate` forwards every other method; only `refresh()` is wrapped
- DI decides whether the app gets the plain or the timed repository — callers can't tell

INTERVIEW TRAP

"Use AspectJ AOP for logging." (last year's advice) — Bytecode weaving on Android means fragile build plugins and behavior nobody can see in the code. Decorators and interceptors do the same job explicitly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. CQRS and event sourcing — on a phone?

Pattern	On the server	On the device
CQRS: separate read and write models	Scale reads independently	You already have it: reads are Room projections as `Flow`, writes are commands in transactions
Event sourcing: state = replay of events	Audit trail, the ledger	Rarely: the server is the source of truth; an unbounded local log costs storage and startup time
Outbox: queue commands, send later	Reliable messaging	Yes — offline writes (Q10)

```
data class OrderRow(val id: String, val title: String)

@Dao
interface OrderDao { // a read model shaped for one screen
    @Query("SELECT id, title FROM orders")
    fun observeRows(): Flow<List<OrderRow>>
}
```

INTERVIEW TRAP

"Event-source the balance on the device: store every deposit and withdrawal, replay them to compute it." (last year's design) — The ledger lives on the server. The app caches the server's balance and queues its own commands.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27

Senior

Server-driven UI · Feature teams & "micro-frontends" · Offline-first & real-time at product level



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Server-driven UI

The server sends **what** to show — a list of components the app already knows. The app decides **how** to render them.

```
@Serializable
sealed interface Block {
    @Serializable @SerialName("header")
    data class Header(val title: String) : Block

    @Serializable @SerialName("carousel")
    data class Carousel(val ids: List<String>) : Block
}

@Composable
fun BlockView(block: Block, onOpen: (String) -> Unit) {
    when (block) {
        is Block.Header -> SectionHeader(block.title)
        is Block.Carousel -> ProductCarousel(block.ids, onOpen)
    }
}
```

Wins	Costs
Change layout and order without an app release	A contract versioned against every app version in the field
Server-side experiments, same layout on Android, iOS, web	Harder testing, previews and accessibility review; cache the last layout for offline



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Surviving unknown blocks

```
val json = Json { ignoreUnknownKeys = true }

// One unknown block must not break the whole screen
fun decodeBlocks(body: String): List<Block> =
    json.parseToJsonElement(body).jsonArray.mapNotNull { el ->
        try {
            json.decodeFromJsonElement<Block>(el)
        } catch (e: SerializationException) {
            null // a block type this app version doesn't know
        }
    }
```

- By default an unknown `type` makes decoding the **whole** list throw — decode element by element
- Send the app version with the request, so the server only sends blocks that version supports

INTERVIEW TRAP

"With SDUI we never need app releases." — Only for layouts built from components the installed app already knows. A new component needs a release. And Google Play forbids downloading executable code (dex, native libraries) outside Play — server-driven layouts are data, not code.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Feature teams and "micro-frontends"

Platform team owns	Feature teams own
Build, CI, convention plugins	Their feature modules, `:api` and `:impl`
Design system and core modules	Screens, ViewModels and data of their domain
Release train, app health dashboards	Their flags, metrics and on-call for their feature
Guardrails: graph rules, Lint, Konsist	Decisions inside their modules (MVVM or MVI)

- **Inverse Conway maneuver:** shape the teams the way you want the architecture to look
- **One binary, one release train:** features merge behind flags and ship on a fixed cadence — deploy and release are decoupled (Day 24)



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What "independent" means in one APK

INTERVIEW TRAP

"Micro-frontends let each team ship on its own release cycle with its own tech stack." (last year's claim) — Not inside one APK. There is one binary, one process, one dependency graph (conflicting library versions resolve to one) and one release. Independence comes from module boundaries and flags.

FOLLOW-UP

"How do you keep the platform team from becoming a bottleneck?" — Inner source: feature teams contribute to core modules through pull requests, the platform team owns review and quality. Paved roads — templates, convention plugins — instead of approvals.

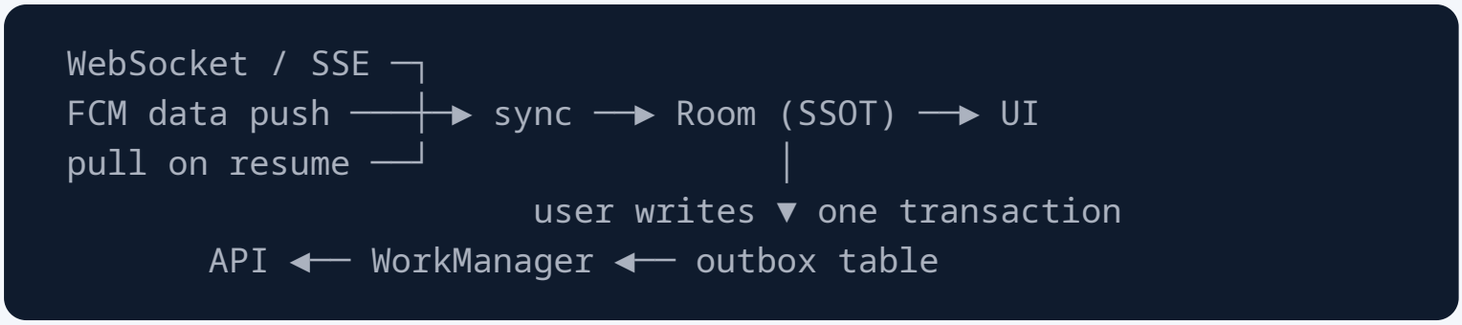
SIGNS THE TEAM STRUCTURE WORKS

A typical feature touches only its own modules. Teams release behind flags without coordinating. Build and review times stay flat as teams are added.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Offline-first and real-time at product level



Offline behavior is a **product decision per feature**, not a library setting:

Feature	Offline read	Offline write	Conflicts
Chat	Cached history	Queued, shown as "sending"	Append-only; the server orders messages
Notes	Yes	Yes	Merge per field; OT or CRDTs for shared text
Payments	Last known status	**No** — online only	The server decides
Feed	Cached pages	—	The server wins



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. One pipeline for everything

```
@Dao
abstract class MessageDao {
    @Insert abstract suspend fun insert(m: MessageEntity)
    @Insert abstract suspend fun enqueue(op: OutboxEntry)

    // Optimistic write + pending command, atomically
    @Transaction
    open suspend fun send(m: MessageEntity, op: OutboxEntry) {
        insert(m.copy(status = Status.SENDING))
        enqueue(op)
    }
}
```

- Real-time channels only **feed the database**; the UI observes Room like everything else
- Unique WorkManager work with a network constraint drains the outbox (Days 14–15); retries carry an idempotency key (Q13)

INTERVIEW TRAP

"For real-time, the UI subscribes to the socket directly." — Now there are two sources of truth, and offline, reconnect and process death each need their own code path.

INTERVIEW TRAP

"Last write wins is the simple default." — With device clocks, skew lets an older edit win. Compare server-assigned versions, and choose the policy per feature.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27



Architect

Decision making & ADRs · Evolving architectures · The app in a distributed system



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Architecture Decision Records

```
# ADR-014: Server-driven layout for the home screen
Status: Accepted · Supersedes ADR-009
Context: Home changes every sprint; many users run
  old app versions; three teams edit home.
Decision: Home is a list of server blocks rendered
  by a client registry. Unknown blocks are skipped.
Options rejected: WebView home; weekly releases.
Consequences:
  + Merchandising ships without an app release
  - Versioned contract, tests on both sides
  - New block types still need an app release
Revisit: if home latency or crash rate regresses
```

- Short, in the repository next to the code, reviewed like code
- **Immutable:** when a decision changes, a new ADR supersedes the old one
- The rejected options and the *why* are what future teams need most



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. How decisions get made

Decision	Process
Reversible and local — a screen's pattern, a library inside one module	The team decides; no ceremony
Hard to reverse or cross-team — sync model, API contract, DI framework, module rules	Written proposal, a spike, review by the affected teams, an ADR
Uncertain	A time-boxed prototype with a success metric, before committing

INTERVIEW TRAP

"An architecture committee approves every proposal." (last year's framework) — It becomes the bottleneck and teams route around it. Decide close to the problem; escalate only irreversible, cross-team decisions — and write them down.

FOLLOW-UP

"How do you compare options?" — Name the qualities that matter here (release speed, offline, build time, team autonomy), score each option against them, and prototype the riskiest assumption first.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Evolving an architecture

Strategy	How	Example
Strangler fig	New code in the new style; old parts replaced one by one until nothing is left	Views to Compose, screen by screen
Branch by abstraction	Interface in front of the old code; build the new one behind it; switch	Old networking stack to Ktor
Parallel run	Run both, compare results, keep using the old one	New pricing or sync logic
Flagged rollout	Switch per user group, watch metrics, roll back instantly	Any risky replacement

```
class SwitchingFeedRepository @Inject constructor(
    private val legacy: LegacyFeedRepository,
    private val modern: OfflineFirstFeedRepository,
    private val flags: FeatureFlags,
) : FeedRepository {
    override fun observeFeed(): Flow<List<Post>> =
        if (flags.isEnabled(Flag.NEW_FEED)) modern.observeFeed()
        else legacy.observeFeed()
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Giving a migration an end

- **Stop new usage first:** ban the old API with a custom Lint check, detekt's `ForbiddenImport` or a Konsist test — otherwise the migration never ends
- **Make progress visible:** percentage migrated per module, on a dashboard
- **Done means deleted:** the old path, the switch and the flag are removed

INTERVIEW TRAP

"Let's rewrite the app from scratch." — Feature work stops while the old app keeps changing, and the rewrite chases a moving target. Incremental migration keeps shipping and can be rolled back at every step.

FOLLOW-UP

"Two patterns in one codebase — isn't that inconsistent?" — Temporarily, yes. That's acceptable if the target is documented in an ADR, new code follows it, and the old pattern has a deletion plan.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The app in a distributed system

- **Old versions live on:** every API change must stay backward compatible — additive fields, versioned endpoints, and a minimum supported version enforced with in-app updates (Day 24)
- **Tolerant reader:** the client ignores unknown fields and maps unknown enum values to a fallback
- **BFF (Backend for Frontend):** an API shaped for the app's screens that aggregates backend services — fewer round trips, app-specific versioning. GraphQL is another answer (Day 12)
- **Eventual consistency is a UI state:** "pending", "syncing", "failed — retry"

```
val json = Json {
    ignoreUnknownKeys = true // new fields from newer servers
    coerceInputValues = true // unknown enum value → default
}

@Serializable
data class OrderDto(
    val id: String,
    val status: OrderStatus = OrderStatus.UNKNOWN,
)
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Retries without duplicates

```kotlin
@Entity(tableName = "outbox")
data class OutboxEntry(
    @PrimaryKey val key: String = UUID.randomUUID().toString(),
    val path: String,
    val body: String,
)

// Every retry of the same command sends the same key
suspend fun HttpClient.send(entry: OutboxEntry) =
    post(entry.path) {
        header("Idempotency-Key", entry.key)
        contentType(ContentType.Application.Json)
        setBody(entry.body)
    }
```

INTERVIEW TRAP

"Retrying a failed POST is safe." — A timeout doesn't tell you whether the server processed the request. Without an idempotency key a retry can charge twice; with one, the server recognizes the repeat and returns the first result.

SENIOR ANSWER IN ONE LINE

"Architecture at scale is about change: make the expensive decisions explicitly and write them down, keep contracts backward compatible, and give every migration an end."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 27 · ARCHITECT

Day 27 — cheat sheet

- Architecture = the decisions that are expensive to reverse. Put the rigor there.
- Conway's law: team boundaries and module boundaries shape each other — design both.
- Model the domain with value classes and invariants; each bounded context owns its model. Money is never a `Double`.
- Commands ask, events report, state describes — "logged in" is state, not a bus event.
- State machines: explicit states and allowed transitions, effects on entry — the value is what they forbid.
- Plugins: extension points + DI multibinding; sort explicitly; no try/catch around composables.
- Cross-cutting concerns: decorators, interceptors, injected collaborators — not base classes or AspectJ.
- SDUI: known components from server data; skip unknown blocks; new components still need a release.
- Offline + real-time: every source writes to the database; writes go through an idempotent outbox.
- Decide with ADRs, migrate with strangler fig + flags, keep APIs compatible with old app versions.

Day 28 is next → Performance at Scale. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700